
Concurrencia y Paralelismo

Bloque I: Concurrencia

Junio 2024

1. Threads que esperan por otros [2 puntos]

Queremos implementar usando mutexes y variables de condición las siguientes funcionalidades:

- a) Todos los threads trabajadores tienen que esperar a que el thread principal llegue a un punto determinado para continuar su ejecución (`barrier_worker()`). El thread principal indica a los threads que ha llegado al punto determinado llamando a la función `barrier_main()`.

```
struct barrier {
    pthread_mutex_t m;
    pthread_cond_t w;
    ...
};

void barrier_worker(struct barrier *b);
void barrier_main(struct barrier *b);
```

- b) Todos los threads tienen que esperar a que el resto de threads lleguen al punto de sincronización. El punto de sincronización se marca con la llamada a `barrier_sync()`. Cuando el último thread llega al punto de sincronización todos continúan.

```
struct barrier {
    pthread_mutex_t m;
    pthread_cond_t w;
    ...
};

void barrier_sync(struct barrier *b);
```

```
int pthread_mutex_lock(pthread_mutex_t *m);
int pthread_mutex_trylock(pthread_mutex_t *m);
int pthread_mutex_unlock(pthread_mutex_t *m);
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mtx);
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_broadcast(pthread_cond_t *cond);
```

Solución:

a)

```
struct barrier {
    pthread_mutex_t m;
    pthread_cond_t w;
    int reached; // Initialized to 0
};

void barrier_worker(struct barrier *b) {
    pthread_mutex_lock(&b->m);
    if(!b->reached)
        pthread_cond_wait(&b->w, &b->m);
    pthread_mutex_unlock(&b->m);
}

void barrier_main(struct barrier *b) {
    pthread_mutex_lock(&b->m);
    b->reached = 1;
    pthread_cond_broadcast(&b->w);
    pthread_mutex_unlock(&b->m);
}
```

b)

```
struct barrier {
    pthread_mutex_t m;
    pthread_cond_t w;
    int n_threads;
};

void barrier_sync(struct barrier *b) {
    pthread_mutex_lock(&b->m);
    b->n_threads--;
    if(b->n_threads > 0)
        pthread_cond_wait(&b->w, &b->m);
    else
        pthread_cond_broadcast(&b->w);
    pthread_mutex_unlock(&b->m);
}
```

2. Secuencia de operaciones [1.5 puntos]

Un array de datos `d` puede ser accedido por varios threads simultaneamente para hacer una operación `do_op` sobre una de sus posiciones. Esta operación tiene efectos secundarios (no es pura). Para proteger el acceso a esos datos existe un array de mutex `d_m`, donde cada posición de `d_m` protege uno de los datos del array.

Vamos a implementar una operación `do_op_seq` que realice una secuencia de operaciones `do_op` especificada por un array que indica, en orden, las posiciones sobre las que hay que operar. Por ejemplo, para hacer la operación sobre las posiciones 1, 5 y 3:

```
int ids[] = {1, 5, 3};
do_op_seq(ids, 3);
```

La secuencia de operaciones no debe realizarse hasta que se hayan bloqueado todas las posiciones afectadas, y debe respetarse el orden indicado por el array de operaciones.

```
void do_op(data d);
void sort(int arr[], int arr_size); // Sort function for int arrays

data d[N];
pthread_mutex_t d_m[N];

void do_op_seq(int ids[], int ids_len) {
    for(int i=0; i < ids_len; i++)
        do_op(d[ids[i]]);
}
```

Implemente la protección de la secuencia de operaciones en `do_op_seq`.

Solución:

```
void do_op(data d);
void sort(int arr[], int arr_size);

data d[N];
pthread_mutex_t d_m[N];

void do_op_seq(int ids[], int ids_len) {
    int sorted_ids[ids_len];

    for(int i=0; i < ids_len; i++)
        sorted_ids[i] = ids[i];

    sort(sorted_ids, ids_len);

    for(int i=0; i < ids_len; i++)
        pthread_mutex_lock(d_m[sorted_ids[i]]);

    for(int i=0; i < ids_len; i++)
        do_op(d[ids[i]]);

    for(int i=0; i < ids_len; i++)
        pthread_mutex_unlock(d_m[sorted_ids[i]]);
}
```

3. Servidor de datos [1.5 puntos]

Escriba un módulo que permita crear procesos servidor con la siguiente interfaz:

- `start()`, que arranca un proceso servidor y devuelve su PID.
- `put(S, Data)`, donde `S` es el PID de un proceso servidor, y `Data` un dato arbitrario. El dato debe quedar almacenado en el servidor.
- `get(S, Proc)`, donde `S` es el PID de un proceso servidor, y `Proc` el PID de un proceso cualquiera. La función debe devolver una lista con todos los datos almacenados por `Proc`.

```
-module(store).  
  
-export([start/0, get/2, put/2]).  
  
start() ->  
    spawn(?MODULE, init, []).  
  
put(S, Data) ->  
    ...  
  
get(S, Proc) ->  
    ...  
  
init() ->  
    ...
```

Solución:

```
-module(store).  
  
-export([start/0, get/2, put/2]).  
  
start() ->  
    spawn(?MODULE, init, []).  
  
put(S, Data) ->  
    S ! {put, Data, self()}.  
  
get(S, Proc) ->  
    S ! {get, Proc, self()},  
    receive  
        {get_reply, L} -> L  
    end.  
  
init() ->  
    loop([]).  
  
loop(L) ->  
    receive  
        {get, Proc, From} ->  
            From ! {get_reply, [Data || {Data, Owner} <- L, Owner == Proc]},  
            loop(L);  
        {put, Data, From} ->  
            loop([Data, From] | L)  
    end.
```
